Smart Search: Solving the 8-puzzle

Write a Python program to solve the 8-puzzle. A solution involves arriving at the final GOAL state given any (random shuffled) initialized set of tiles, where the GOAL state is defined as:

```
-------------
| 0 | 1 | 2 |
-------------
| 3 | 4 | 5 |
-------------
| 6 | 7 | 8 |
-------------
```

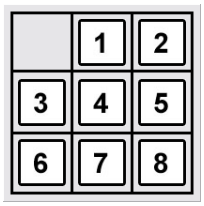
```
# possible data structure in Python for the GOAL
# the tile zero(0) indicates the OPEN slot
GOAL = [ [0, 1, 2], [3, 4, 5], [6, 7, 8] ]
```

Input [optional] (`stdin`, keyboard):

A user enters a starting board configuration (or for experiments, a random board is used). *(Note: see the method `initializePuzzleBoard()` in the `Board` class and the note below on detecting an invalid starting board).*

Output (`stdout`, console):

A final path of tile moves from the initial board to the GOAL board. The final path output *must* reflect a realistic set of tile moves (that is, your output can not print boards of your solutions that jump around the search tree; you *can* of course, while debugging, print the boards that you *expand*). The final printout must be an accurate set of tile moves. (Note: you will undoubtedly want to dump more output to the console during debugging; just make sure *on submission* that your console output is *only* a final solution path.)




```
>>> [evaluate EightPuzzle_Main.py]

SEARCH ALGORITHM: Best-First     HEURISTIC: Manhatten Distance = (sum of misplaced tiles)

DONE!! 8-Puzzle solved in 4 moves.

STARTING BOARD
-------------
| 3 | 1 | 2 |
-------------
| 4 | 7 | 5 |
-------------
| 6 | 8 | 0 |
-------------

Move:  1
-------------
| 3 | 1 | 2 |
-------------
| 4 | 7 | 5 |
-------------
| 6 | 0 | 8 |
-------------

Move:  2
-------------
| 3 | 1 | 2 |
-------------
| 4 | 0 | 5 |
-------------
| 6 | 7 | 8 |
-------------

Move:  3
-------------
| 3 | 1 | 2 |
-------------
| 0 | 4 | 5 |
-------------
| 6 | 7 | 8 |
-------------

Move:  4
-------------
| 0 | 1 | 2 |
-------------
| 3 | 4 | 5 |
-------------
| 6 | 7 | 8 |
-------------

YES!
```

Output (`.csv` Excel-ready file):

Your program, while solving the problem, should dump statistical information to a comma-separated value (.csv) output file. Your stats (e.g., see Table 1) should include the current Board# tried (that is, the latest board popped off (*expanded*) from your Queue of boards to try), as well as the sizes of your Queue of Boards and the Visited List (for DFS, BFS, Best-First) or the size of the Expanded List of Boards (for UC or A*). Timing information would also be valuable.

Board Number Tried	Size of Queue	Size of Visited List
0	1	1
1	2	3
2	3	5
3	5	8
4	6	10

Table 1. Using a best-first search, the initial lines of output of one solution while solving the 8-puzzle. The sizes of the data structures, Queue and Visited List, are shown. (Missing is a column that shows the "wall" time (seconds) (so far) for each line of output).

After you implement a random board generator, your final report should also report other metrics on your solution space, e.g., over 1000 trials, such as shown in Table 2.

Algorithm	Average Maximum Size of Queue	Average Runtime (sec)
Best-First: $h_1(n)$ = # of misplaced tiles		
Best-First: $h_2(n)$ = Manhatten Distance		
Depth-First Search		

Table 2. Comparison of three algorithms for 1000 trials of randomly generated (valid) starting 8-puzzle boards.

Algorithm

To start, implement the Best-First Search strategy discussed in class. You might think of this as the A* algorithm where all edges (moves) have the same cost (e.g., +1). Your program should use two different heuristics for "estimating" the distance between a given board and the GOAL board; the two heuristics are (i) the number of tiles out of place and (ii) the Manhatten Distance = [the sum of tile misplacements] (as discussed in class).

You *must* implement (at least the) `class BoardClass()`. The starterKit includes another `.py` file as a (beginning) test-driver. The Board class should override the relational operators so that a list of BoardClass objects can be sorted according to the current heuristic used. (Note: you must implement different versions of these relational operators once you change to the alternate heuristic; you may want to make two different versions of your app, e.g., v2.0 uses Manhattan Distance.)

Note: not all starting boards are solvable, thus if/when you generate random board configurations you really must have a method that checks if your starting configuration is even possible. (An impossible board means your little nephew probably took his jackknife to your tiles, popped out some of the tiles, and put them back in to mess with your head ☺). There is a straightforward function to determine if a starting board can reach our Goal state; the algorithm is shared here: http://www.8puzzle.com/8_puzzle_algorithm.html. Of course, you should implement this method, probably called alongside a method that generates a *valid* random board.

Experiment(s)

So, your program solves the 8-puzzle … (*yawn*) … um, I mean "*I am soooo proud of you*!" Now the important (empirical) work begins. Run a suite of starting game boards (randomly generated) and keep statistics. (Note: plan carefully! Many of these runs may take a long time?) Your program should produce results that will appear in a Final Report (a professional looking report) that includes the sections: (a) Summary of the program/task and a concluding paragraph that mentions what else will be found in the report (a succinct summary is important; it should be a high-level description; it should *not* for instance start talking about user input, data structures, etc), (b) Methods: description and number of input set(s); summary of Algorithm(s) used; special cases? (c) Sample Output: a sample solution path (obviously, a small run); (d) Results: the total runtime for finding each solution in your input set(s); Table(s) and Figure(s)/plots of your results. Remember: your Final Report *is* the most significant part of your submission!

Submission: .zip of a folder (named with YourLastName.zip *that produces upon unzip* a folder with YourLastName) containing the file(s): `_README.txt` , *documented* source code (documentation following PEP-8 style, including, docstrings and type hinting for all methods), any input sets used (if any), a .py file that implements **unit testing**, a sample output (e.g., Result_Using_BestFirstSearch.txt), and your Final Report (.pdf). Please submit a hardcopy of your report in class on Monday, Oct. 19.

Grading

- **Average (C):** you get the algorithm working on at least some algorithm; show *correct* results, and success on various trials; A final report is submitted.
- **Above Average (B):** you implement both heuristics for Best-First Search; you use and document Python objects and run experiment(s); you trap bad boards; you time experimental runs; your report includes Tables and Figures (plots) of runtime(s). (Note: these Tables and Figures could be produced elsewhere, e.g., Excel or Sheets, and the images are copied into and included in your report).
- **Superior (A): [I am open to suggestions, but assuming you have finished the above]** you implement a competing algorithm, e.g., one of the brute force algorithms (e.g., DFS or BFS) or better yet, the A* algorithm. Your report compares the differences, of course, averaged over many (randomly assigned starting boards) input trials.